

EX NO : 13

DATE : 02-09-26

Digit Classification using ANN

AIM:

To construct, train, and evaluate an Artificial Neural Network (ANN) using the MNIST dataset to classify handwritten digits (0–9), evaluate training vs. validation metrics to detect potential overfitting, and run inference on new input digit images.

CODE:

```
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf
from keras.models import Sequential
from keras.layers import Dense, Flatten
from PIL import Image

(X_train, y_train), (X_test, y_test) = tf.keras.datasets.mnist.load_data()

X_train_norm = X_train / 255.0
X_test_norm = X_test / 255.0

model = Sequential([
    Flatten(input_shape=(28, 28)),
    Dense(128, activation='relu'),
    Dense(64, activation='relu'),
    Dense(10, activation='softmax')
])

model.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)

history = model.fit(
    X_train_norm, y_train,
    epochs=10,
    batch_size=32,
    validation_split=0.1,
    verbose=1
)

test_loss, test_acc = model.evaluate(X_test_norm, y_test, verbose=0)
print(f"\nTest Loss: {test_loss:.4f}")
print(f"Test Accuracy: {test_acc * 100:.2f}%")

plt.figure(figsize=(12, 4))

plt.subplot(1, 2, 1)
plt.plot(history.history['accuracy'], label='Train Accuracy')
plt.plot(history.history['val_accuracy'], label='Val Accuracy')
plt.title('Model Accuracy')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.legend()

plt.subplot(1, 2, 2)
```

```

plt.plot(history.history['loss'], label='Train Loss')
plt.plot(history.history['val_loss'], label='Val Loss')
plt.title('Model Loss')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.legend()

plt.tight_layout()
plt.show()

def predict_custom_image(image_path, model):
    img = Image.open(image_path).convert('L')
    img = img.resize((28, 28))
    img_array = np.array(img)
    if np.mean(img_array) > 127:
        img_array = 255 - img_array
    img_norm = img_array / 255.0
    input_tensor = np.expand_dims(img_norm, axis=0)
    predictions = model.predict(input_tensor)
    predicted_digit = np.argmax(predictions[0])
    confidence = np.max(predictions[0]) * 100
    plt.imshow(img_norm, cmap='gray')
    plt.title(f'Predicted Digit: {predicted_digit} ({confidence:.2f}% Confidence)')
    plt.axis('off')
    plt.show()
    return predicted_digit

```

OUTPUT:

```

Epoch 1/10
1688/1688 ————— 5s 3ms/step - accuracy: 0.9264 - loss: 0.2510 - val_accuracy: 0.9690 - val_loss: 0.1119
Epoch 2/10
1688/1688 ————— 4s 2ms/step - accuracy: 0.9678 - loss: 0.1048 - val_accuracy: 0.9755 - val_loss: 0.0908
Epoch 3/10
1688/1688 ————— 4s 2ms/step - accuracy: 0.9776 - loss: 0.0715 - val_accuracy: 0.9743 - val_loss: 0.0923
Epoch 4/10
1688/1688 ————— 5s 3ms/step - accuracy: 0.9829 - loss: 0.0550 - val_accuracy: 0.9790 - val_loss: 0.0765
Epoch 5/10
1688/1688 ————— 5s 2ms/step - accuracy: 0.9859 - loss: 0.0433 - val_accuracy: 0.9800 - val_loss: 0.0806
Epoch 6/10
1688/1688 ————— 5s 2ms/step - accuracy: 0.9884 - loss: 0.0356 - val_accuracy: 0.9758 - val_loss: 0.0899
Epoch 7/10
1688/1688 ————— 4s 2ms/step - accuracy: 0.9905 - loss: 0.0288 - val_accuracy: 0.9773 - val_loss: 0.0760
Epoch 8/10
1688/1688 ————— 4s 2ms/step - accuracy: 0.9914 - loss: 0.0245 - val_accuracy: 0.9773 - val_loss: 0.0854
Epoch 9/10
1688/1688 ————— 6s 3ms/step - accuracy: 0.9925 - loss: 0.0209 - val_accuracy: 0.9772 - val_loss: 0.0934
Epoch 10/10
1688/1688 ————— 4s 2ms/step - accuracy: 0.9934 - loss: 0.0205 - val_accuracy: 0.9767 - val_loss: 0.1035

Test Loss: 0.1016
Test Accuracy: 97.44%

```

